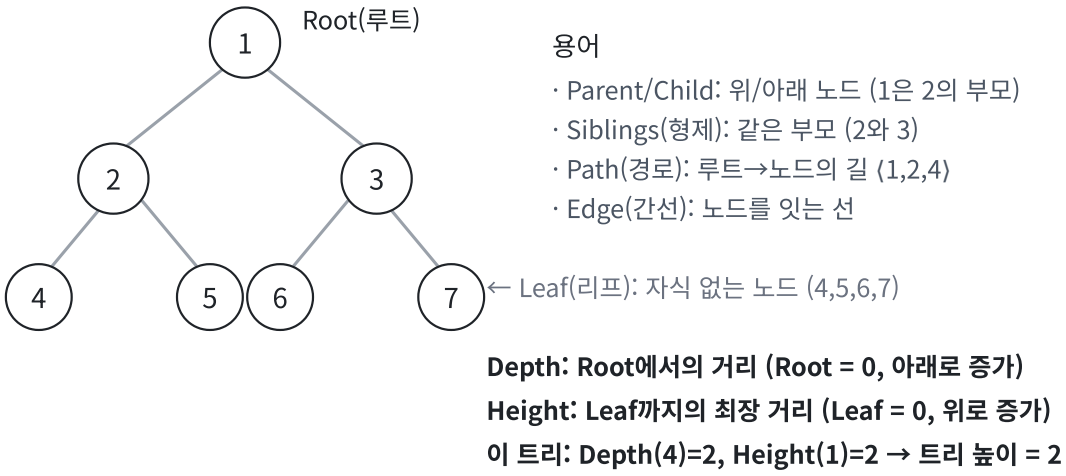
CSED233 · 데이터구조 · 포스텍

정리본 5. Trees

트리 용어 · Depth vs Height · 이진 트리 3가지 특수형 · 배열/연결 구현.

1. Tree(트리) — 계층적 자료구조

Tree(트리): 노드(Node)와 간선(Edge)의 모음으로, 부모-자식 관계로 조직된 계층적 구조. 루트를 제외한 모든 노드는 정확히 하나의 부모를 가진다.



같은 트리를 두 기준으로 본다: Depth는 위에서부터, Height는 아래에서부터

Depth와 Height를 헷갈리지 말 것

- **Depth(깊이)**: Root에서 그 노드까지의 거리. Root는 항상 0.
- **Height(높이)**: 그 노드에서 Leaf까지의 최장 거리. Leaf는 항상 0.
- **트리의 높이 = Root의 Height.**

2. Binary Tree(이진 트리)와 특수형

Binary Tree(이진 트리): 각 노드가 최대 2개의 자식을 가지는 트리. Left/Right가 명시적으로 구분된다(일반 트리와 다른 점).

```
struct TreeNode {
  int val;
  TreeNode* left;  // 왼쪽 자식
  TreeNode* right; // 오른쪽 자식
};
```

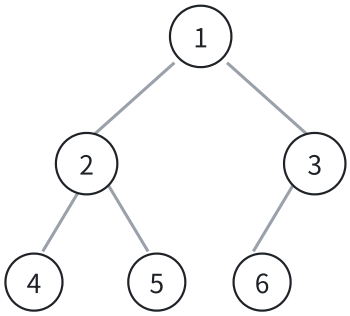
2.1 Complete Binary Tree(완전 이진 트리)

마지막 레벨을 제외한 모든 레벨이 완전히 채워지고, 마지막 레벨은 왼쪽부터 차례로 채운 트리. 순서대로 채워지므로 **배열로 구현 가능**하다(포인터 불필요).

배열 인덱스 관계식 (1-indexed, 0번 칸은 비움)

$$\text{parent}(i) = \lfloor i/2 \rfloor, \quad \text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1$$

검증: 노드 4의 부모 = 배열[⌊4/2⌋] = 배열[2] = 2 ✓ · 노드 2의 왼쪽 = 배열[4] = 4 ✓



레벨 1까지 완전 · 마지막 레벨은 왼쪽부터 (3의 오른쪽 자리는 비어 있음)

배열 표현: 노드 번호 = 인덱스

—	1	2	3	4	5	6
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Complete Binary Tree → 배열 그대로 대응

2.2 Proper(0 또는 2개 자식) / Full(모든 레벨 충족)

- **Proper(정식 이진 트리)**: 각 노드가 자식을 0개 또는 2개 가짐 (1개는 금지).
- **Full Binary Tree(포화 이진 트리)**: 모든 레벨이 완전히 채워진 트리. 높이 h이면 노드 수는:

$$n = 2^{h+1} - 1$$

예: 높이 2 → $2^3 - 1 = 7$개 노드.



왼쪽은 모든 내부 노드가 자식 2개, 오른쪽은 위반

2.3 Balanced Binary Tree(균형 이진 트리)

여기서는 AVL 방식의 균형 조건을 사용한다: 모든 노드에서 왼쪽/오른쪽 서브트리의 높이 차이 ≤ 1인 트리. 높이가 O(log n)으로 억제된다. BST의 순서 조건까지 만족하면 검색이 O(log n)으로 보장된다 (일반 이진 트리의 검색은 순서 조건이 없어 O(n)일 수 있음). 비균형(한쪽으로만 긴) 트리는 높이가 O(n)까지 커져 성능이 저하된다 — 그래서 AVL/Red-Black Tree가 자동 균형을 잡는다(강의 10).

3. 구현: 배열 vs 연결

	배열 기반	Linked 기반
가능한 트리	Complete에 특히 효율적 (다른 형태도 가능하나 빈 공간 낭비 큼)	모든 형태의 Binary Tree
관계 표현	인덱스 계산 (⌊i/2⌋, 2i, 2i+1)	left/right 포인터
장점	포인터 불필요, Random Access O(1)	임의 형태 가능, 유연
단점	모양 제약	포인터 오버헤드, Retrieve O(n)

3.1 배열 기반 예시

```
int tree[MAX];  // 1-indexed (0번 인덱스는 안 씀)
tree[1] = 10
tree[2] = 5,  tree[3] = 15
```

```
tree[4] = 3,  tree[5] = 7,  tree[6] = 12,  tree[7] = 20
//        10(1)
//       /    \
//     5(2)    15(3)
//    /  \    /  \
//  3(4) 7(5) 12(6) 20(7)
```

3.2 Linked 기반 실전 예제

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left  = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left  = new TreeNode(4);
    root->left->right = new TreeNode(5);
    //        1
    //       / \
    //      2   3
    //     / \
    //    4   5
    return 0;
}
```

이 트리의 순회(강의 6에서 자세히): Preorder 1 2 4 5 3 · Inorder 4 2 5 1 3 · Postorder 4 5 2 3 1

4. 핵심 정리

- 트리 연산의 시간복잡도는 높이에 의존: $O(h)$. 균형이면 h = $O(\log n)$, 비균형이면 h = $O(n)$.
- Depth는 위에서(Root=0), Height는 아래에서(Leaf=0) 센다.
- Complete → 배열 구현 가능 · Proper/Full → 자식 0/2개, 모든 레벨 충족 · Balanced → 높이 차 $\leq$ 1.

- 배열 구현은 Complete에 특히 효율적 (다른 형태도 가능하나 공간 낭비 큼), Linked 구현은 모든 형태 가능.

5. Quiz

Q1. Depth(깊이)와 Height(높이)의 차이는?

> 힌트

세는 방향이 반대다: 하나는 Root에서, 하나는 Leaf에서.

> 정답

개념은 힌트 참고.

정답: Depth는 위에서 아래로(Root=0), Height는 아래에서 위로(Leaf=0) 센다. 트리의 높이는 Root의 Height.

Q2. Complete Binary Tree가 배열로 구현되는 이유는?

> 힌트

빈틈없이 순서대로 채워지면 위치를 "계산"할 수 있다.

> 정답

개념은 힌트 참고.

정답: 순서대로 채워져 있어 인덱스 계산(⌊i/2⌋, 2i, 2i+1)으로 부모-자식 관계를 바로 알 수 있어 포인터가 필요 없기 때문. 메모리도 효율적.

Q3. 배열 구현에서 인덱스 10에 있는 노드의 부모는?

> 힌트

parent(i) = ⌊i/2⌋.

> 정답

개념은 힌트 참고.

① 공식 적용: ⌊10/2⌋ = 5.

② 배열[5]에 있는 노드가 부모다.

정답: 인덱스 5의 노드 (⌊10/2⌋ = 5).

Q4. Full Binary Tree와 Complete Binary Tree의 차이는?

> 힌트

Full은 "모든 레벨 충족" 규칙, Complete는 "채우는 순서" 규칙, Proper은 "자식 개수" 규칙.

> 정답

개념은 힌트 참고.

정답: Full은 모든 레벨이 가득 찬 트리, Complete는 레벨 순서대로 빈틈없이 채운 트리. (Full은 Complete이지만 Complete이라고 Full은 아니다. 자식 1개가 금지된 트리는 Proper.)

Q5. 비균형 Binary Tree의 문제점은?

> 힌트

연산 비용은 높이 h에 비례한다. 한쪽으로 길어지면 h는?

> 정답

개념은 힌트 참고.

① 한쪽으로만 자라는 최악의 경우 높이 h = n.

② 검색·삽입 비용이 O(log n)에서 O(n)으로 저하.

③ 해결: AVL/Red-Black Tree로 자동 균형(강의 10).

정답: 높이가 O(n)까지 커져 검색도 O(n)으로 저하된다 → 균형 트리가 필요해지는 이유.